

Отчет по лабораторной работе №2

Выполнил: Тюгаев Н.П.

Описание проекта

Разработан автоматизированный конвейер для обработки видеоконтента с последующей генерацией и локализацией субтитров. Система принимает видеофайлы через Telegram-бота, выполняет распознавание речи на английском языке, переводит субтитры на русский и встраивает их в исходное видео. В качестве технологического стека выбраны `n8n` для оркестрации процессов, специализированный микросервис на базе Whisper для распознавания речи, `Ollama` для машинного перевода и `Docker` для контейнеризации компонентов.

Архитектура системы

Проект реализован в виде микросервисной архитектуры с четким разделением ответственности между компонентами:

1. **n8n** — центральный оркестратор, управляющий всем workflow обработки видео
 - Базовый образ: `docker.n8n.io/n8nio/n8n:latest` на Alpine Linux
 - Дополнительно установлен `ffmpeg` для работы с медиафайлами
 - Обрабатывает входящие запросы от Telegram, координирует выполнение этапов пайплайна
2. **autosub** — выделенный сервис для преобразования речи в текст
 - Основан на `python:3.10-slim` (Debian) для обеспечения совместимости с PyTorch
 - Реализован как REST API на FastAPI
 - Использует библиотеку `auto-subtitle`, построенную на OpenAI Whisper
 - Архитектура сервиса модульная: основной код в `app.py`, конфигурация вынесена в `config.py`, зависимости управляются через `requirements.txt`
 - Включает валидацию входных данных и обработку ошибок на уровне API
3. **Ollama** — локальный сервер для выполнения машинного перевода
 - Использует модель `llama3` для перевода английских субтитров на русский
 - Работает через HTTP API, интегрирован в workflow `n8n`
4. **ngrok** — туннель для обеспечения HTTPS-соединения
 - Предоставляет публичный HTTPS-адрес для webhook от Telegram
 - Позволяет работать с локальным экземпляром `n8n`

Технические детали реализации

Структура Docker-конфигурации

n8n/Dockerfile: Расширение базового образа n8n установкой `ffmpeg` для обработки видео и аудио.

autosub/Dockerfile: Многослойная сборка с оптимизацией кэширования:

- Установка системных зависимостей (`git`, `ffmpeg`, `wget`)
- Копирование `requirements.txt` и установка Python-пакетов
- Установка `auto-subtitle` из GitHub и PyTorch (CPU-версия)
- Копирование исходного кода приложения (`app.py` , `config.py`)
- Настроен `healthcheck` для мониторинга работоспособности сервиса

docker-compose.yml: Оркестрация всех сервисов с настройкой:

- Общие тома для обмена файлами между контейнерами
- Healthchecks для всех критичных сервисов (`n8n`, `autosub`, `ollama`)
- Зависимости между сервисами с условиями готовности
- Переменные окружения для конфигурации

Модульная структура сервиса autosub

Сервис спроектирован с разделением ответственности:

- `app.py` — основная логика приложения: обработка HTTP-запросов, валидация, выполнение транскрибации
- `config.py` — централизованное хранение констант (порты, пути, поддерживаемые модели Whisper)
- `requirements.txt` — управление зависимостями проекта

Функции валидации входных данных проверяют существование файлов, корректность путей, поддерживаемые модели. Обработка ошибок реализована с возвратом информативных HTTP-ответов.

Последовательность выполнения workflow

Пайплайн обработки видео состоит из следующих этапов:

1. **Telegram Trigger** — получение видеофайла от пользователя через бота
2. **Write Binary File** — сохранение входящего видео в `/files/input.mp4`
3. **Extract Audio** — извлечение аудиодорожки командой `ffmpeg` в формат WAV (`/files/audio.wav`)

4. **Generate Subtitles** — HTTP POST-запрос к `autosub:5000/transcribe` для генерации английских субтитров
5. **Read SRT** — чтение созданного файла субтитров
6. **Translate** — отправка содержимого SRT в Ollama с промптом для перевода на русский с сохранением таймкодов
7. **Save Translated SRT** — сохранение переведенных субтитров в `/files/out_ru.srt`
8. **Burn Subtitles** — встраивание субтитров в видео через `ffmpeg (hardsub)`
9. **Read Output Video** — чтение обработанного видео
10. **Send Video** — отправка результата пользователю в Telegram
11. **Cleanup** — удаление временных файлов независимо от результата выполнения

Обработка ошибок реализована через Error Trigger в n8n, который уведомляет пользователя и выполняет очистку ресурсов.

Встреченные проблемы и их решения

1. Динамический URL ngrok

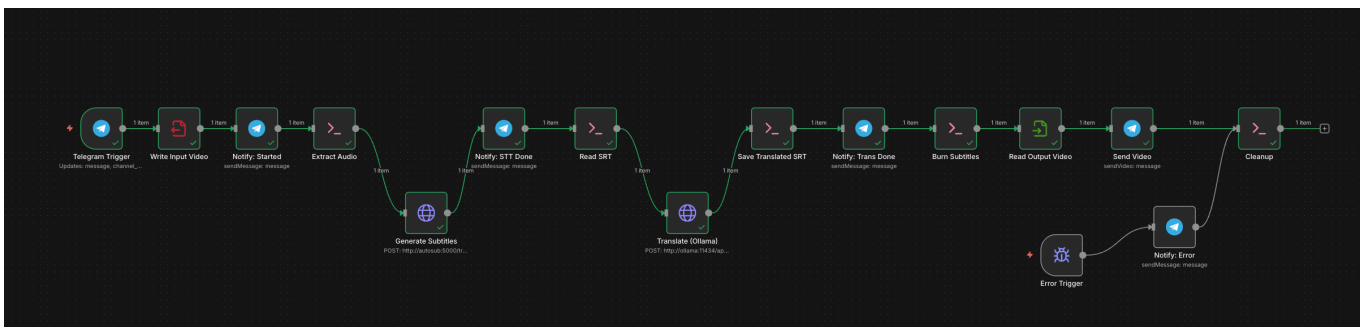
- Проблема: При использовании бесплатного плана ngrok публичный URL меняется при каждом перезапуске контейнера
- Решение: Реализован процесс ручной регистрации webhook через Telegram Bot API при изменении адреса. URL проверяется через веб-интерфейс ngrok и обновляется в конфигурации.

2. Обеспечение отказоустойчивости

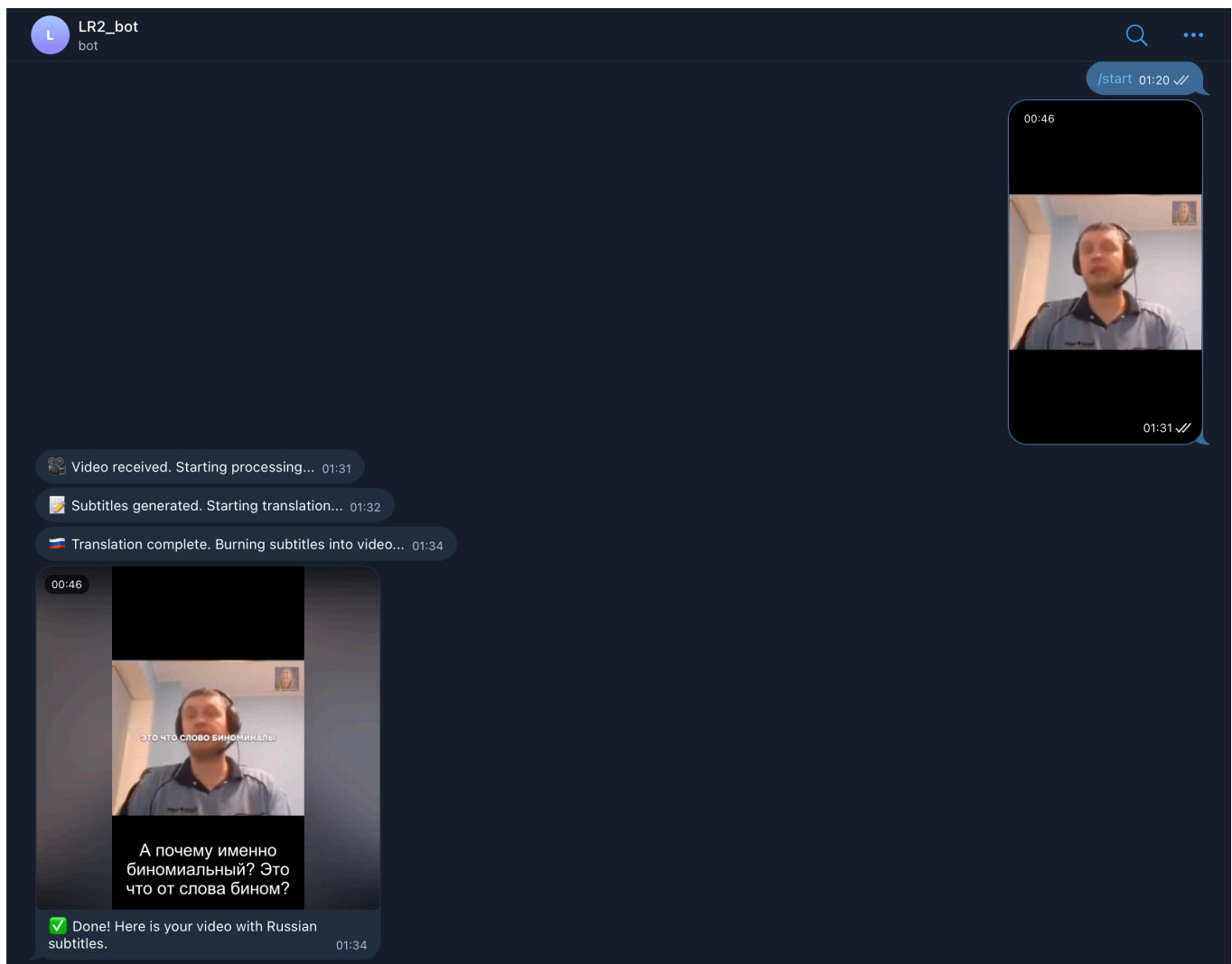
- Проблема: Необходимость контроля работоспособности сервисов
- Решение: В docker-compose настроены healthchecks для всех критичных компонентов, что позволяет автоматически определять состояние сервисов и корректно обрабатывать зависимости при запуске.

Скриншоты

Пайплайн обработки в n8n



Результат работы бота в Telegram



Заключение

Реализованная система демонстрирует эффективный подход к автоматизации обработки видеоконтента с использованием современных технологий. Микросервисная архитектура обеспечивает гибкость и масштабируемость решения. Модульная структура кода, разделение конфигурации и бизнес-логики, а также встроенные механизмы валидации и обработки ошибок делают систему надежной и поддерживаемой. Использование локальных моделей (Whisper, Llama) позволяет обрабатывать данные без отправки в облачные сервисы, что важно с точки зрения конфиденциальности.

Инструкция по запуску

1. Подготовка окружения:

- Создать файл `.env` на основе `.env.example`
- Заполнить необходимые переменные (`NGROK_AUTHTOKEN`, `WEBHOOK_URL` и др.)

2. Запуск контейнеров:

```
docker-compose up -d
```

3. Загрузка модели Ollama:

```
docker exec -it ollama_service ollama pull llama3
```

4. Настройка Telegram webhook:

- Получить текущий ngrok URL: `http://localhost:4040/status`
- Остановить n8n: `docker-compose stop n8n`
- Обновить `WEBHOOK_URL` в `.env` файле
- Зарегистрировать webhook (можно через браузер):

```
https://api.telegram.org/bot<токен_вашего_бота>/setWebhook?url=
<ваш_ngrok_url>/webhook/telegram-trigger
```

- Запустить n8n: `docker-compose up -d n8n`

5. Настройка workflow в n8n:

- Открыть веб-интерфейс: `http://localhost:5678`
- Импортировать workflow из файла `n8n/pipeline.json`
- Настроить credentials для Telegram API
- Активировать workflow

После выполнения этих шагов система готова к работе. Можно отправлять видеофайлы боту в Telegram для обработки.